Behique CI: demo recording kit

Run of show, clean state, and what you can and cannot say. Prepared 2026-10-29, updated 2026-10-30 (new beat 10 and the claims about other apps, the crawler and the sandbox). The live FreshRSS beats (1 to 8) are unchanged. The PDF next to this file is a reading copy of the same content.

Read this first

- The hackathon's video length limit and judging criteria were never verified. This kit has three cuts: full about 5:39, standard about 3:00, short about 1:48. Check the real limit before choosing.
- Every line you say maps to something the pipeline has shown (section 4). If a number on screen differs from this script, say what the screen shows.
- Section 5 is a list of things you must not say. Read it before the take.
- Beat 10 (other apps) is NOT live. It shows saved results from 2026-10-30, one run per repo, and each run took 5 to 40 minutes in the sandbox. Say so on camera.
- The demo page's 'Coming soon' list and the Cause explainer status were brought up to date on 2026-10-30. If a line on the page disagrees with what you know, say what is true.
- Ship, Hold and AI guesses are permanent in a report (it is append-only). Run the prep script before every take.

1. Clean state

Done on 2026-10-29. Reports are moved to an archive, not deleted, so this is reversible.

Report	What it is	Use
20261029T060213Z_1.29.1-vs-1.30.0	Share case, fresh, no decision, 0 of 3 AI guesses used	Fallback for the live run; has the About card
20261029T060341Z_1.29.1-vs-1.30.0	Scroll case (#9285), fresh, no decision	Shown on camera in beat 8 as 'run earlier'
20261029T051556Z_1.29.1-vs-1.30.0	Older share case that already holds 1 AI guess	Backup if the AI call fails; no About card (predates scenarios)
20261029T051746Z, 20261029T055632Z, 20261029T055925Z	Duplicate runs	Moved to out/evidence-archive/20261029T065103Z/

The live run on camera adds one more fresh share report at the top of the picker.

Before each take (archives everything except the backup, then makes one fresh report of each case):

```
node demo/prepare.mjs --generate --keep 20261029T051556Z_1.29.1-vs-1.30.0
```

Without `--generate` it only archives. It refuses to run if a check or a hand-test build is still up. It prints each report's scenario, decisions, AI guesses, whether the record is intact, whether the model key is set, and whether the demo server is running.

Pre-flight checklist:

- Run the prep script (below). Wait for it to finish; it takes about 3.5 minutes with --generate.
- Open http://127.0.0.1:8790 in a normal browser window about 1280 pixels wide at 100% zoom, not the narrow side pane. Dark or light both work.
- Close other tabs, turn on Do Not Disturb, and check that nothing private is on screen. Never open .env.
- Do one rehearsal of beats 3 to 7 before the real take. It will add a decision and an AI guess to that report, so run the prep script again afterwards.
- The AI step needs network access and the Meta key. The prep script prints 'set' or 'MISSING'; it never prints the key.

- Keep the terminal out of view. If you show it, run nothing that prints environment variables.

2. Run of show (full cut)

Spoken text is written for about 2.5 words a second. Square brackets are cues, not words.

#	Time	Beat	On screen	Say	Evidence
1	0:00 to 0:20	Hook	Top of the demo page: logo and title.	When a new version of a program ships, the hard question is: what changed, and does it matter? Behique CI compares an old version with a new one. It only counts something as proven when it has a test that can be run again, and a person makes the final call.	Demo page header and the idea strip. Every claim the code writes must cite saved evidence or it is refused (evidence.mjs rules, 8 passing tests). AI guesses are labelled as leads, not proof.
2	0:20 to 0:35	The setup	Scroll to The idea, then See it work. Leave 'Share menu link' selected.	My test subject is FreshRSS, a real open-source news reader: old version 1.29.1, new version 1.30.0. I will start with a known case, the Share menu link.	Scenario list on the page (scenarios.mjs).
3	0:35 to 1:35	Live run	Press Run the check. Let the timeline fill in. Put a caption 'sped up' on screen during the test runs and the search, and keep the elapsed timer visible.	I press Run. It starts both versions from clean installs and records over twelve hundred facts about the page after each click. Some change by themselves, like timestamps and security tokens, so it loads the old version twice to find those, and ignores them. A third recording must show zero differences, or it stops. Two values changed between the versions; the rest are new or moved elements. It writes a test for the main change and runs it three times on each version, each on a fresh install: three passes on the old, three failures on the new. Then it searches the 227 code changes between the versions, testing the middle one and halving the list each time. Eight steps, then it double-checks its answer.	Share report: 174 of 1,267 facts masked (13.7%); control run 0 differences; 2 changed, 18 added, 21 moved; test 3 of 3 and 3 of 3; 227 changes, 8 steps, verified 3 of 3 and 3 of 3. Real run time observed 88 to 107 s.
4	1:35 to 2:10	The result	Scroll to the verdict: the old and new values, then the code change card. Hover, do not click yet.	Result: a behaviour change, proven and traced to one change. The old version copies a normal link. The new one copies an encoded one. The change is titled 'Security: apply encodeURIComponent to share template'. The change just before it passes the test, and this one fails it, three times out of three each. Whether that matters is not Behique's call.	Same case: facts-diff.json (data-url was https://example.com/..., now https%3A%2F%2F...), bisect.json (commit 1b7178a). Clipboard copy independently checked in headless Chromium on both builds.
5	2:10 to 2:45	The report	Click 'Open the full report and record Ship or Hold'. Show the intact banner, What the check proved, and click one evidence link (R1.spec.js) to show the saved test. Show the empty 'only a lead' section.	Here is the full report. Everything under 'what the check proved' was measured by code, and each item links to the saved evidence it rests on, like the exact test that ran. The record is chained, so changing any earlier entry is detected. Possible causes and AI guesses get their own section, because they are never proof.	Report page. Tamper detection is covered by unit tests and a test on a copy. Do not edit a real report on camera, and do not promise the banner will turn red (that display was not visually tested).

#	Time	Beat	On screen	Say	Evidence
6	2:45 to 3:20	The AI step	In 'only a lead', show the Ask an AI card: read the projected cost aloud, open 'See exactly what will be sent', then press the button and wait a few seconds.	The AI step is the only one that uses a model, and only when I press the button. It shows the projected cost first: under one cent, never more than two, at an estimated price. The model sees only the two changed values and that one code change. It offers a guess, labelled as a guess, and code checks that the line it quotes really is in the change. [Read what it says, then use the matching line below.]	hypothesis.mjs: Meta muse-spark-1.3, reasoning low, 4,000 output token cap, 3 guesses per report. First real call cost about $0.0062 against a $0.0074 projection (price is an unverified estimate). The answer wording varies between calls.
7	3:20 to 3:40	The human decides	Scroll to Your decision. Type a short note such as 'demo take'. Press Hold (or Ship). Point at the decision on record.	Behique never ships or holds a release. I do. I press Hold and say why. It is saved to the record with the time, and if I change my mind, the earlier decision stays on the record.	Tested in the browser on a throwaway copy: Ship then Hold, newest counts, earlier kept, record intact. This will stay on the report for good, so re-run the prep script before the next take.
8	3:40 to 4:20	A second case	Use the report picker to open the scroll report (run earlier). Show the About this case card, then the verdict and the old and new values.	This second case I ran earlier to save time. Upstream issue 9285 was reported as a regression: scrolling changed in 1.30.0. The maintainers could not reproduce it as described, the reporter closed it, and I found no fix; the behaviour was changed on purpose. So Behique does not call it a bug. It proves the behaviour really differs: the page does not scroll on the old version, and jumps 4,033 pixels on the new one. It traces that to the change that was made on purpose. Whether that is a bug is the decision.	Scroll report: 412 of 2,944 facts masked; exactly 1 changed fact (window.scrollY 4033 to 0); test 3 of 3 and 3 of 3; first bad change aaae9ee (#8870), 8 steps, verified. Upstream: #9285 closed as completed by its reporter, maintainers could not reproduce it as described; no fix commit found in 1.30.1 to 1.30.4.
9	4:20 to 4:52	Where the AI fits	Back on the demo page: the Where the AI fits table.	The explorer is a separate piece. It looks at an app it has never seen and reports bugs. In one scored run it found 13 of 27 planted bugs on a small test app, and it raised false alarms on both of its clean runs, so its reports are leads. On FreshRSS I wrote the click steps by hand: in two runs the explorer never opened an article, and it found the Share menu only after a one-line hint. The live run above still uses click steps I wrote by hand; the next beat shows what replaces them.	bench/final-results.json (Meta: 13 of 27 detected, 2 of 2 clean runs had a false alarm). Explorer on FreshRSS: 0 of 2 without a hint, 1 of 1 with one (out/explorer).
10	4:52 to 5:22	Other apps, no hand-written steps	Show the saved results (docs/STATUS.md section 'Checks on real repos', or the out/check-bench tables). Caption 'saved runs, not live'. Point at the six rows.	The live run used steps I wrote. I also built a crawler that finds click paths itself, with no model: it only reads, it never types and it blocks every request but GET. On FreshRSS it reached the same answer with no hand-written steps: the same two values, the same change. Then I ran the whole check on six other open-source projects, inside Docker on a server, with a database next to the app when it needs one. Five gave confirmed changes, one showed no change. Confirmed only means the test passes on the old version and fails on the new, every time. It does not mean the change matters, and I have no ground truth for these apps. To make it work, the check now learns the noise a repeated recording shows, which hides some facts, so a real change inside those could be missed.	FreshRSS crawl check (docs/STATUS.md, Wave 3): 1b7178a in 8 steps, verified 3 of 3 and 3 of 3. Six repos (out/check-bench 2026-10-30, sandbox on hardac2): html5-boilerplate no change (crawl found only the landing page); excalidraw 5 of 5; nextjs-blog 5 of 5; umami 3 of 3; todomvc 5 of 5; vitepress 1 of 1, a version label (the final row of each repo is in out/check-bench/2026-10-30T20-00-27, 21-32-25 and 22-08-29; the first full run was 1 confirmed, 1 no change, 4 failed before the noise fix). Masked by learning: nextjs-blog 1,223 facts, todomvc 66, vitepress 40. Run times 1.3 to 38 minutes. Planted-changes bench: 8 of 8 visible changes found, 7 of 8 confirmed, 0 false positives on a comment-only change.

#	Time	Beat	On screen	Say	Evidence
11	5:22 to 5:39	Close	The coming-soon list (explorer without a hint, apps behind a login, a CI pipeline), then the logo.	Built: the comparison, the test, the search for the exact change, a tamper-evident record, and a report where a person decides. Not built: the explorer finding flows unaided, logins and seeded data, a ground truth for what matters on other apps, and hooking this into a CI pipeline. That is Behique.	docs/STATUS.md (built and not built lists).

Total 5:39.

Beat 6, after the guess appears, say one of these:

- If the guess says the change encodes the value twice or encodes the whole URL: 'It suggests the change encodes the value again, which matches the two facts we saw. That fits the evidence. It is still one guess from one model, and I still have to judge it.'
- If the guess does not match the evidence: 'This guess does not match what we measured. That is exactly why it sits under only a lead, and not under what the check proved.'
- If the page shows 'the line it points to was not found': read that warning out loud and say it lowers your trust in the guess.

Time-lapse note: a real run takes about 90 to 107 seconds. Roughly 0 to 13 s is recording and comparing, 13 to 40 s is the test and its six runs, and 40 to 100 s is the search. Speed up the last two parts in the edit, caption it 'sped up', and keep the page's elapsed timer visible so nobody thinks it was instant.

3. Shorter cuts

Standard cut, about 3:00. beat 1 Hook (0:00 to 0:15); beat 2 The setup (0:15 to 0:25); beat 3 Live run (0:25 to 1:10); beat 4 The result (1:10 to 1:30); beat 5 The report (1:30 to 1:50); beat 6 The AI step (1:50 to 2:15); beat 7 The human decides (2:15 to 2:25); beat 8 A second case (2:25 to 2:50); beat 11 Close (2:50 to 3:00).

Short cut, about 1:48. beat 1 Hook (0:00 to 0:10); beat 3 Live run (0:10 to 0:40); beat 4 The result (0:40 to 0:55); beat 6 The AI step (0:55 to 1:15); beat 7 The human decides (1:15 to 1:23); beat 8 A second case (1:23 to 1:38); beat 11 Close (1:38 to 1:48).

In the standard and short cuts, trim each kept beat to its first sentences; keep the honest lines (beat 8's 'so Behique does not call it a bug', and beat 11's 'Not built'). Beat 10 (other apps) is only in the full cut.

4. What you can claim, and where it comes from

You can say	Source
It compares an old and a new version and only counts something as proven when a replayable test shows it.	Every claim written by code cites saved evidence or is refused (evidence.mjs, 8 tests). The comparison, test and search have no AI in them. The AI step is separate and its output is labelled a guess.
FreshRSS 1.29.1 against 1.30.0, a real open-source app.	Adapter checks out both tags from the real repository.
Over twelve hundred facts recorded, about 14% change on their own and are ignored; a control run shows 0 differences.	Share report 20261029T060213Z_1.29.1-vs-1.30.0: 174 of 1,267 (13.7%), control 0. Scroll report: 412 of 2,944 (14.0%), control 0.
Share case: 2 values changed, 18 new, 21 moved.	Share report summary line.

You can say	Source
The test passes 3 of 3 on the old version and fails 3 of 3 on the new, each on a fresh install.	Reproduced-failure claim in both reports; test-runs.json.
227 code changes searched in 8 steps; the answer verified 3 of 3 and 3 of 3.	bisect.json in both reports (227 changes, 8 steps, verified).
Share case first bad change: 1b7178a, 'Security: apply encodeURIComponent to share template (#9146)'.	bisect.json. Upstream: #9146 is a merged pull request; #9302 is the issue titled 'Regression with sharing links'.
Scroll case: the old version does not scroll (0 px), the new one jumps 4,033 px; first bad change aaae9ee (#8870).	Scroll report claims and bisect.json.
#9285 was closed by its reporter, the maintainers could not reproduce it as described, and I found no fix.	GitHub issue 9285 and its comments; git log of 1.29.1..1.30.4 shows no fix. Say 'I found no fix', not 'there is no fix'.
A full run takes about a minute and a half to under two minutes.	Observed 88.3, 91.7, 102.6 and 106.8 s on four runs.
The AI step uses Meta's muse-spark-1.3, sends only the two changed values and one code change, and costs under a cent.	hypothesis.mjs; first real call about $0.0062 against a $0.0074 projection. The price is an unverified estimate; say 'estimated'.
The model's quoted line is checked to be really in the code change.	quoteIsInDiff, covered by hypothesis.test.mjs (offline).
A person's Ship or Hold is saved with the time; earlier decisions stay.	Tested in the browser on a throwaway copy.
Editing an earlier entry in a record is detected.	Unit tests, and a test on a copy of a real report. Not visually tested on the page.
The explorer found 13 of 27 planted bugs in one scored run, and raised false alarms on both clean runs.	bench/final-results.json, Meta. One run, one model: no percentages, no 'accuracy'.
On FreshRSS the explorer missed the Share menu 2 of 2 times without a hint and found it 1 of 1 time with one.	out/explorer: three saved runs.
A crawler finds click paths with no model (read only: GET and HEAD only, nothing typed). On FreshRSS, with no hand-written steps, the check reached the same two values and the same first bad change 1b7178a (8 steps, verified 3 of 3 and 3 of 3).	docs/STATUS.md Wave 3; `node src/release/check-app.mjs freshrss 1.29.1 1.30.0 --bisect`. FreshRSS is the app the crawler was tuned on.
The full check ran on six other open-source repos in Docker on a server (with a Postgres sidecar for umami): five gave confirmed changes, one (html5-boilerplate) no change.	out/check-bench/2026-10-30T20-00-27, 21-32-25, 22-08-29; docs/STATUS.md 'Checks on real repos'. One run per repo, and the first full run was 1 of 6 before the noise fix. Say 'confirmed' only as 'passes on the old version and fails on the new, every time'.
Repos run in a sandbox: no outside network for the app, install scripts off unless approved, memory and cpu limits.	docs/sandbox.md; the policy is in src/release/sandbox/docker-args.mjs. It is containers, not Firecracker or gVisor, and it is not a claim that it is unbreakable.
Planted changes: 8 of 8 visible changes detected, 7 of 8 confirmed as a test, 0 false positives on a comment-only change.	bench/planted-bench.mjs, out/check-bench/planted-latest.json. A small generated app, not a real one. The unconfirmed one is an added `disabled` attribute.

5. Do not say

- That the AI explorer found the Share menu bug, or any bug, by itself on FreshRSS. The explorer only found the Share menu after a hint. The live run uses hand-written steps. The crawler (no model) reached the same answer later, on the app it was tuned on.

- That #9285 is a bug, a confirmed regression, or something that was fixed. It is a reproducible behaviour difference that was changed on purpose; a person decides.
- That the AI found the cause. The search found the commit, deterministically. The AI only offered a guess, and 'correct' is not something this pipeline has shown.
- That the explorer has no false positives, or any accuracy percentage. It was one run on one model with 13 of 27 detected and false alarms on both clean runs.
- That it works on any app, or that it found real bugs in the six repos. It has run on FreshRSS (two scenarios, one version pair), a small planted-bug app, and six open-source repos with one run each and no ground truth. Apps behind a login or needing seeded data are out of reach (umami's crawl reached 2 pages). A canvas app gets the picture check, not the crawl.
- That 'confirmed' on those repos means the change matters. Several are trivial: a version label, a React-generated id `_r_1_` to `_r_2_`.
- That the noise learning is free. Facts it masks can hide a real change: nextjs-blog 1,223, todomvc 66, vitepress 40.
- That the sandbox is a hard security boundary. Do not name Firecracker or gVisor: not built.
- That it plugs into CI, GitHub Actions or any pipeline. That is not built; 'CI' is only in the name.
- That the whole flow is autonomous. A person approves a new app's set-up and every command a model proposes, and makes the ship or hold decision. FreshRSS still uses an adapter and the live demo hand-written steps; other repos use a profile the model proposed and code verified.
- Any price as a fact. Costs are estimates at an unverified third-party price. Do not quote a total spend.
- That this matches the hackathon's track or judging criteria, or how it compares with past winners. The official rules were never verified.
- That the integrity banner will turn red on camera, and do not edit a real report to show it.
- Anything from the .env file. Never show or read out keys.

6. If something goes wrong

Problem	What to do
The live run fails or hangs	Say 'I will show a run from earlier', open the share report (20261029T060213Z_1.29.1-vs-1.30.0) from the picker and continue from beat 4. Check the terminal for the cause afterwards.
The AI call fails (network, key, limit)	Do not retry more than once. Open the backup report (20261029T051556Z_1.29.1-vs-1.30.0), which already holds one AI guess, and say it was recorded earlier. It has no About card because it predates scenarios.
The AI says something odd	Good. Use the 'does not match' line. Never edit or hide a guess.
'Another check is running'	A hand-test build or an earlier run is still up. Run node demo/prepare.mjs to see which port is busy.
The demo page does not load	Start the 'demo' preview in the app, or run node demo/server.mjs, then open http://127.0.0.1:8790.
A number on screen differs from this script	Say what the screen shows. The script's numbers are 'about'.

7. After recording

- Run `node demo/prepare.mjs --generate --keep 20261029T051556Z_1.29.1-vs-1.30.0` again so the next take starts clean. Decisions and AI guesses from this take go to the archive with their reports.
- Keep the archive folder if you want the take's reports as a record.
- If you spent AI guesses during rehearsal, each was about three-quarters of a cent at the estimated price.